

Home

About



Robotics and ROS Learning

YouTube 5K

ROBOTICS

Jacobian

Jacobian is Matrix in robotics which provides the relation between joint velocities ($\dot{q}$) & end-effector velocities ($\dot{X}$) of a robot manipulator.

If the joints of the robot move with certain velocities then we might want to know with what velocity the endeffector would move. Here is where Jacobian comes to our help. The relation between joint velocities and end-effector velocities is given as below,

$$\dot{X} = J\dot{q}$$

where,

$\dot{q}$ is the column matrix representing the joint velocities. Size of the this matrix is $n \times 1$. 'n' is the number of joints of the robot.

$\dot{X}$ is the column matrix representing the end-effector velocities. Size of this matrix is $m \times 1$. 'm' is 3 for a planar robot and 6 for a spatial robot.

J is the Jacobian matrix which is a function of the current pose . Size of jacobian matrix is $m \times n$.

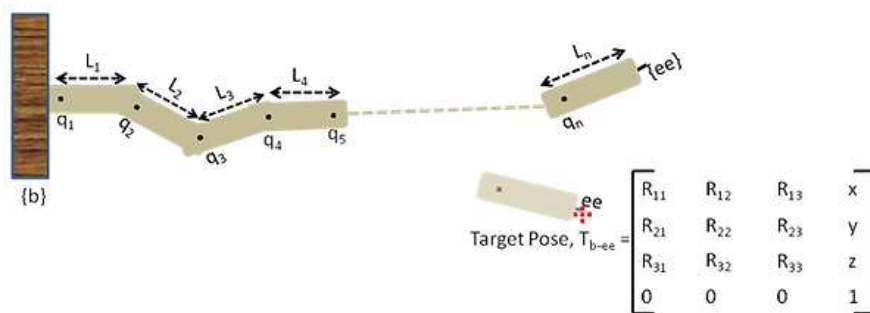


Fig. 1 A spatial robot with n-joint/n-DOF

Let me write the above equation ($\dot{X} = J\dot{q}$) in expanded matrix form for a spatial robot for better understanding.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\alpha} \\ \dot{\beta} \\ \dot{\gamma} \end{bmatrix}_{6 \times 1} = \begin{bmatrix} J_{11} & J_{12} & \dots & \dots & J_{1n} \\ J_{21} & J_{22} & \dots & \dots & J_{2n} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ J_{61} & \dots & \dots & \dots & J_{6n} \end{bmatrix}_{6 \times n} * \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \vdots \\ \dot{q}_n \end{bmatrix}_{n \times 1} \quad \text{----- (*)}$$

Time to understand the Jacobian matrix.

Columns of the Jacobian matrix are associated with joints of the robot. Each column in the Jacobian matrix represents the effect on end-effector velocities due to variation in each joint velocity.

Which means, the first column represents the effect of joint1 velocity ($\dot{q}_1$) on end-effector velocities ($\dot{X}$), second column is associated with joint2 velocity ($\dot{q}_2$) and similarly nth column is effect of nth joint velocity ($\dot{q}_n$) on end-effector velocities .

Hence the number of columns in the Jacobian matrix is equal to the number of joints in the manipulator.

If we closely observe the $\dot{X}$ matrix, it has two parts. The first three elements of the end-effector velocity matrix $[\dot{x}, \dot{y}, \dot{z}]$ are linear velocities [rate of change of position] and the last three elements $[\dot{\alpha}, \dot{\beta}, \dot{\gamma}]$ are the angular velocities [rate of change of orientation] in (x,y,z) direction respectively.

Similarly, **rows** of the Jacobian matrix can also be split into two part. The first three rows are associated with linear velocities of end-effector and the last three rows are associated with the angular velocities of end-effector due to change in velocities of all the joints combined.

Hence we can call the upper part of the Jacobian matrix as Linear velocity Jacobian (J_v) and the lower part as Angular velocity Jacobian (J_ω).

$$J = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix}_{6 \times n}$$

Now let's see how to derive a Jacobian matrix of robotic manipulator.

Methods to derive J_v and J_ω are different. We will find them separately and later combine to get our final Jacobian matrix.

Finding J_v :

We all know from our elementary physics class that velocity is nothing but the first order derivative of position. Since J_v is related to linear velocities of the end-effector due to joint velocities, we can get the J_v by derivating the position functions for x, y and z of the end-effector w.r.t joint variables $[q_1, q_2, q_3, \dots, q_n]$ as shown below.

$$J_v = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \frac{\partial x}{\partial q_2} & \frac{\partial x}{\partial q_3} & \dots & \dots & \dots & \frac{\partial x}{\partial q_n} \\ \frac{\partial y}{\partial q_1} & \frac{\partial y}{\partial q_2} & \frac{\partial y}{\partial q_3} & \dots & \dots & \dots & \frac{\partial y}{\partial q_n} \\ \frac{\partial z}{\partial q_1} & \frac{\partial z}{\partial q_2} & \frac{\partial z}{\partial q_3} & \dots & \dots & \dots & \frac{\partial z}{\partial q_n} \end{bmatrix}_{3 \times n}$$

I guess by now a question would be running in your mind. Where from in this world we will get the functions for x, y and z.

I will give you a hint. The hint is Forward Kinematics.

Still confused, click here for the [answer](#).

Yes you have got it right. The last column of the Transform matrix T_{b-ee} will provide us the functions for the position of end-effector.

We are half way through finding the Jacobian Matrix.

Lets find the second part.

Finding J_ω :

J_ω is related to the angular velocities of the end-effector. Again from our high school physic, we know that angular velocity (ω) is pseudo vector and is given by the product of axis of rotation ($\hat{\omega}$) and rate of rotation ($\dot{\theta}$) about the axis as shown in the below fig 2.

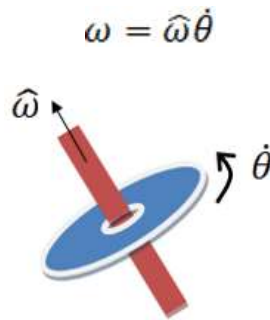


Fig. 2 A disk rotating about the axis $\hat{\omega}$ with velocity $\dot{\theta}$ radians/sec

$\hat{\omega}$ is a unit vector representing the axis of rotation in 3D space. It is written in the below form,

$$\hat{\omega} = a_1 \hat{x} + a_2 \hat{y} + a_3 \hat{z}$$

Thus this unit vector can be represented as a 3x1 matrix as shown below,

$$\hat{\omega} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

Henceforth, angular velocity can be represented in matrix form as below

$$\text{Angular velocity, } \omega = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \cdot \dot{\theta}$$

I have represented angular velocity in vector form to show you a similarity with Jacobian matrix in (*) equation above and make it simpler to find the J_ω matrix.

If we observe (*) equation, rate of rotation of all joints (joint velocities $\dot{q}_1, \dot{q}_2, \dots, \dot{q}_n$) are already present in the $[\dot{q}]$ matrix. So the only missing component to find the angular velocities of the end-effector is the axis of rotation information of each joint. This information, i.e., joint axes of all joints, is what J_ω matrix is all about.

Now there is a twist here in finding the joints axes. All joint axes in J_ω are w.r.t base frame, not from the local frame.

Confused....??

Don't worry the below example will make things much clearer.

Look at the below fig 3. It is n link spatial manipulator. I have taken 3 joints i, j and k to explain you how to find the joint axes w.r.t base frame.

As the first job, I have attached frames to the links and lets say I have found the forward kinematics also.

The frames $\{i\}$, $\{j\}$, $\{k\}$ are the local frames and $\{b\}$ is the base frame to joints i, j and k respectively.

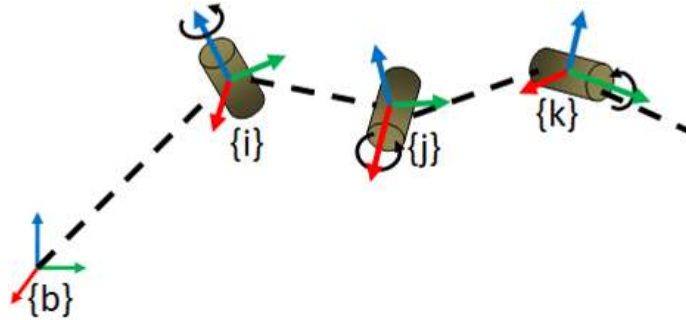


Fig. 3 A n-link Spatial manipulator with $\{b\}$, $\{i\}$, $\{j\}$, $\{k\}$ focused

From fig 3 it is clear that joint i is rotating about z-axis of the local frame $\{i\}$, joint j about x-axis of local frame $\{j\}$ and joint k about y-axis of local frame $\{k\}$. So the joint axes of joint i, j and k within their local frames are represented as below.

$$\hat{\omega}_i^i = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Axis of joint i
w.r.t frame $\{i\}$

$$\hat{\omega}_j^j = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Axis of joint j
w.r.t frame $\{j\}$

$$\hat{\omega}_k^k = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Axis of joint k
w.r.t frame $\{k\}$

Finding the axis of the joint in the local frames looks easy right.

But we need the joint axis w.r.t to base frame $\{b\}$ and finding this is not that straight forward. We have to pre-multiply the local joint axes $\hat{\omega}_i^i$, $\hat{\omega}_j^j$, $\hat{\omega}_k^k$ with the rotation matrices R_{bi} , R_{bj} , R_{bk} respectively to get the joint axes w.r.t to base frame.

$$\hat{\omega}_i^b = R_{bi} * \hat{\omega}_i^i$$

Axis of joint i
w.r.t frame $\{b\}$

$$\hat{\omega}_j^b = R_{bj} * \hat{\omega}_j^j$$

Axis of joint j
w.r.t frame $\{b\}$

$$\hat{\omega}_k^b = R_{bk} * \hat{\omega}_k^k$$

Axis of joint k
w.r.t frame $\{b\}$

Looks tough right...?

But don't worry, we need not do the above multiplications to find the joint axes w.r.t frame $\{b\}$, there is a trick to find them easily.

These joint axes information is already present in one of the first three columns of the transform matrices T_{bi} , T_{bj} and T_{bk} (T_{bi} , T_{bj} and T_{bk} matrices would have been already derived while finding the forward kinematics). We just need to know the right column to look at. The right column information is provided by the axis of joint in local frame.

If the axis of rotation of joint in local frame is x, then first column of T_{bi} matrix is joint axis w.r.t base frame. Similarly, second column of T_{bj} for y-axis and third column for z-axis [Ignore the 0]

Note: To understand frame assignments and finding Transformations, go through [Rigid Body Transformations](#)

So we can get the joint axes $\hat{\omega}_i^b$, $\hat{\omega}_j^b$, $\hat{\omega}_k^b$ from T_{bi} , T_{bj} and T_{bk} as shown below.

$$T_{bi} = \begin{bmatrix} X & X & \hat{\omega}_i^b & X \\ X & X & a_2 & X \\ X & X & a_3 & X \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{bj} = \begin{bmatrix} \hat{\omega}_j^b & X & X & X \\ a_2 & X & X & X \\ a_3 & X & X & X \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{bk} = \begin{bmatrix} X & \hat{\omega}_k^b & X & X \\ X & a_2 & X & X \\ X & a_3 & X & X \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Isn't that simple.

Since we already know all the transform matrices from base to end-effector while finding the forward kinematics of our robot, therefore we have all joint axes w.r.t base frame information beforehand.

So all we need to do is group all these joint axes from joint-1 to joint-n [$\hat{\omega}_1^b$, $\hat{\omega}_2^b$, ..., $\hat{\omega}_n^b$] to get the J_ω matrix.

$$J_\omega = [\hat{\omega}_1^b \quad \hat{\omega}_2^b \quad \hat{\omega}_3^b \quad \dots \quad \hat{\omega}_n^b]_{3 \times n}$$

Finally....!! we have J_v and J_ω . So we just have to stack them to get the complete Jacobian matrix.

$$J = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \frac{\partial x}{\partial q_2} & \frac{\partial x}{\partial q_3} & \dots & \dots & \frac{\partial x}{\partial q_n} \\ \frac{\partial y}{\partial q_1} & \frac{\partial y}{\partial q_2} & \frac{\partial y}{\partial q_3} & \dots & \dots & \frac{\partial y}{\partial q_n} \\ \frac{\partial z}{\partial q_1} & \frac{\partial z}{\partial q_2} & \frac{\partial z}{\partial q_3} & \dots & \dots & \frac{\partial z}{\partial q_n} \\ \hat{\omega}_1^b & \hat{\omega}_2^b & \hat{\omega}_3^b & \dots & \dots & \hat{\omega}_n^b \end{bmatrix}_{6 \times n}$$

Hurray.....!!!!!! we now know the in and out about the Jacobian matrix

A solved example to find the Jacobian matrix of a 6-DOF spatial manipulator is given [below](#)

But wait, the real question is still unanswered. How are we going to solve the inverse kinematics using Jacobian matrix.

So lets learn about the Jacobian Inversion Method.

Jacobian Inversion Method:

This method of inverse kinematics can be applied in two ways based on the type of joint actuators.

Method 1: For the robots with velocity controlled joint actuators.

Method 2: For the robots with position controlled joint actuators.

Before going to learn about the above two methods, lets re-frame the question and see what all values we know and what needs to be derived.

Q) Lets say we have n- link spatial robot as shown in below fig 4. The robot is at some random pose at this moment. Now the task is to move the end-effector of the robot to a given goal pose, T_g w.r.t base.

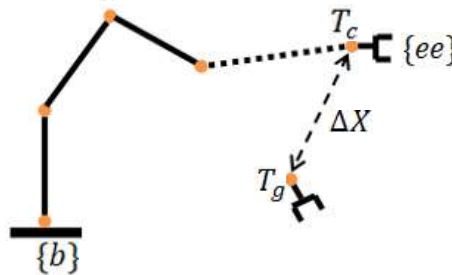


Fig. 4 A n-link Spatial manipulator

We can know joint position values $[q_1, q_2, q_3, \dots, q_n]$ of the current pose through the sensors (joint encoders) present at each joint. Hence we can get the Transformation matrix of the end-effector w.r.t base using FK. Lets call this transform matrix as T_c

Since we have worked so hard to understand Jacobian matrix. By now we can derive Jacobian matrix. So we also know J matrix as the function of joint position values $[q_1, q_2, q_3, \dots, q_n]$.

T_g is given goal pose, so we know T_g as well.

Let me put all the known things in one frame.

1. Current Joint positions, $q = [q_1, q_2, q_3, \dots, q_n]^T$
2. Current end-effector position and orientation, T_c . From this T_c matrix we can get the $X_c = [x_c, y_c, z_c, \alpha, \beta, \gamma]^T$
3. Goal position and orientation of end-effector T_g (or) $\dot{X}_g$
4. Jacobian matrix [J] as function of current joint values.

Now lets see how to accomplish this task using Jacobian Inversion Method.

Method 1:

Since Jacobian gives a direct relation between end-effector velocities ($\dot{X}$) and joint velocities ($\dot{q}$), the solution to inverse kinematics problem for a robot accepting velocity commands (radians/sec) is straight forward. All we need to do is to compute the end-effector velocities and Jacobian inverse. Finally find the joint velocities using below equation

$$\dot{q} = J^{-1} \dot{X}$$

Once we found the joint velocities, feed them to the joint actuators. In given time the end-effector will reach the desired goal pose. Once the goal pose is reached send 0 velocity commands to all the joint actuators to stop further movement of the robot.

That's all, our task of getting the end-effector to the desired pose is completed.

Lets write an algorithm for this method.

Step 1: Find ΔX

$$\Delta X = X_g - X_c$$

Step 2: Convert ΔX into velocity $\dot{X}$.

$$\dot{X} = p \cdot \Delta X$$

where p is simply a proportionality constant which decides the speed of the end-effector to reach the target pose.

Step 3: Using the current joint position values $[q_1, q_2, q_3, \dots, q_n]$, find the numerical value of Jacobian matrix.

Step 4: Find the pseudo inverse (J^+) of the Jacobian matrix. (why pseudo inverse is discussed in the section Singularities).

Step 5: Compute $\dot{q}$ using the below equation and feed the velocity commands to the joint actuators.

$$\dot{q} = J^+ \dot{X}$$

Step 6: Finally, track the pose of the end-effector and stop the joint actuator's movement once the goal pose is reached (i.e., $\Delta X = 0$).

Method 2:

This method is applied to the robots whose joint actuators accept position commands (radians). Which means we have to find new q_{new} values (instead of $\dot{q}$) and feed them to joint actuators. Hence the Jacobian velocities equation cannot be used directly for this robots.

So we have to now make a relation between end-effector displacement and joint positions instead of their velocities. The same Jacobian equation can be used for displacement domain also but it only holds good for small displacements.

$$\delta q = J^{-1} \delta X$$

Whats "small displacement"?? There is no general recipe for that. It depends on the physical structure of the robots.

So we make ΔX a small displacement by multiplying it with some fraction value 'f' and using the above equation we get δq . This δq is added to the q to get the q_{new} values. Now the q_{new} values are given to joint actuators. Hence the end-effector move little closer to the goal pose.

Again ΔX is computed with the new T_c & given goal T_g and further q_{new} values are computed in the same way. This is an iterative method. In each iteration end-effector gets closer-n-closer to the goal pose. Hence this is repeated until the goal pose is reached (i.e., $\Delta X = 0$).

Algorithm for this method

Step 1: Find ΔX .

$$\Delta X = X_g - X_c$$

Step 2: Multiply ΔX with some fraction 'f' to make the displacement small

$$\delta X = f \cdot \Delta X$$

where f is a fractional value and it is empirical.

Step 3: Using the current joint position values $[q_1, q_2, q_3, \dots, q_n]$, find the numerical value of Jacobian matrix.

Step 4: Find the pseudo inverse (J^+) of the Jacobian matrix. (why pseudo inverse is discussed in the section Singularities).

Step 5: Compute δq using the below equation.

$$\delta q = J^+ \delta X$$

Step 6: Add δq to the current joint positions to get new joint position values

$$q_{new} = q + \delta q$$

Step 7: Feed the new joint positions q_{new} to the joint actuators and find the new T_c and X_c .

Step 8: Repeat 1 to 7 until $\Delta X = 0$.

Finally.....!!! we now know what is a Jacobian and how it is used to solve the inverse kinematics problem.

But we are not done yet. We only know the bright side of Jacobian method. Now lets talk about the dark side.

Problems with Jacobian Method:

Finding IK using this method involves matrix inversion. As we all know matrix inversion is not always easy and may not be possible in some cases. There are two such cases in this method where we would be facing trouble to find the inverse of J matrix.

1. When the Jacobian matrix is **not a square matrix**.

Can you guess for which robots the Jacobian matrix is not square..?

For the robots which has number of joints less than or greater than 6 but not exactly 6. i.e., when the robot is under-actuated or a redundant robot, the Jacobian matrix is not square.

2. When the robot is at **Singularity**.

There can be some pose of the robot where inverse may not be possible. This pose is known as singular configuration.

At singular configuration, Jacobian matrix loses its Rank, determinant of Jacobian becomes zero and inverse does not exist. Which physically means that the robot has lost a DOF. This usually happens when the end-effector is at the edges of the workspace i.e., when the robot is fully stretched.

However being in a singularity is not as bad as being very close to the singularity. When the robot is near to the singular configuration, it starts to behave abnormally when using this method.

This is because when the robot is approaching to singular configuration, the Jacobian inversion method will compute and produce larger joint velocities or Δq which is not acceptable. Asking for some finite movement in end-effector space can result in very very large, potentially infinite, movement in joint space which can have unpredictable results.

So while controlling our robot we have to make sure that our robot does not go into singular configuration and even more importantly we have to be careful not to approach singularities.

In Mathematics there is a solution to every problem. Luckily we have a common solution for both the problems mentioned above.

Solution to the Problems:

The solution is using **Pseudo inverse (J^+)** of the Jacobian matrix obtained by the Moore-Penrose matrix inversion instead of J^{-1} .

1. This method uses **Singular value decomposition** to find the inverse of a non square matrix which is the solution to the first problem.

2. The inverse of a singular or non-invertible matrix is also possible with this method. And additionally the joint velocities or the Δq computed using pseudo inverse won't allow any additional movement towards the singularity but will allow any movement that doesn't get us any closer to the singularity. This solves our second problem of singularities.

Ta-Daa.....!! So we can happily use the Jacobian inversion method to find the IK of the robot without any worries.

But I know you are still worried of one thing. That is how to find the Pseudo inverse of the Jacobian matrix. Isn't it..?

Well we are not going to discuss about the derivation of J^+ from J matrix. Because, in practice we don't have to compute this by hand. Since the pseudo inverse is a very commonly used concept in linear algebra, almost any linear algebra library will have a function to compute it.

For example in python the pseudo inverse can be found using below api in numpy lib.

```
 $J^+ = \text{numpy.linalg.pinv}(J, e)$ 
```

Phew.....!!! This ends our topic on Jacobian. Hoping you have understood everything about the Jacobian and enjoyed learning new things, I'll take leave. See you in the next topic.

Example 3:

In this example we find the Jacobian matrix of the 6-DOF robot shown in fig 12.

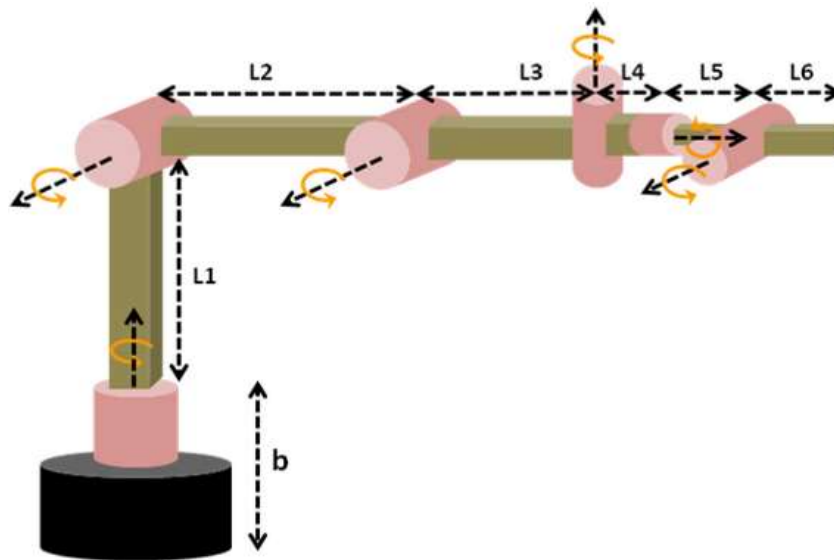


Fig. 5 6-DOF Spatial manipulator

As discussed in Jacobian Technique method, the no. of columns in Jacobian matrix depends on the no. of DOF of the robot. So no. of columns for our 6-DOF manipulator are 6.

Since this robot operates in the spatial workspace and it is a fully actuated robot, the no. of rows are also 6.

Therefore Jacobian for this manipulator is 6X6 square matrix.

The Jacobian matrix is derived using the Transformation matrix.

$$J = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \frac{\partial x}{\partial q_2} & \frac{\partial x}{\partial q_3} & \frac{\partial x}{\partial q_4} & \frac{\partial x}{\partial q_5} & \frac{\partial x}{\partial q_6} \\ \frac{\partial y}{\partial q_1} & \frac{\partial y}{\partial q_2} & \frac{\partial y}{\partial q_3} & \frac{\partial y}{\partial q_4} & \frac{\partial y}{\partial q_5} & \frac{\partial y}{\partial q_6} \\ \frac{\partial z}{\partial q_1} & \frac{\partial z}{\partial q_2} & \frac{\partial z}{\partial q_3} & \frac{\partial z}{\partial q_4} & \frac{\partial z}{\partial q_5} & \frac{\partial z}{\partial q_6} \\ R_{b1}[:3] & R_{b2}[:1] & R_{b3}[:1] & R_{b4}[:3] & R_{b5}[:2] & R_{b6}[:1] \end{bmatrix}$$

So lets start to find each Transformation matrices $T_{b1}, T_{b2}, \dots, T_{b-ee}$.

As we know to find the Transformations, the first step is frame assignment.

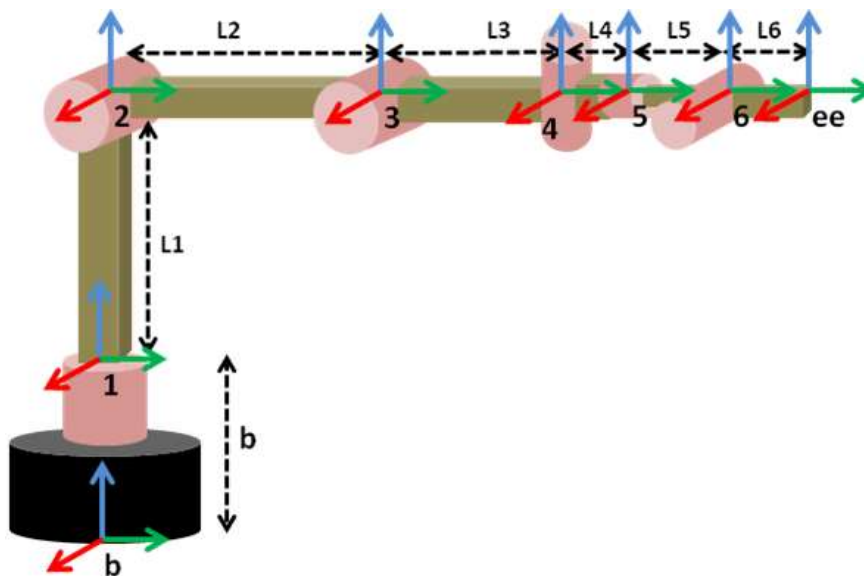


Fig. 6 Frames attached to our 6-DOF manipulator

As you have guessed, the next step is to write down the adjacent transformation matrices.

$$\begin{aligned}
 T_{b1} &= \begin{bmatrix} c_1 & -s_1 & 0 & 0 \\ s_1 & c_1 & 0 & 0 \\ 0 & 0 & 1 & b \\ 0 & 0 & 0 & 1 \end{bmatrix} ; \quad T_{12} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_2 & -s_2 & 0 \\ 0 & s_2 & c_2 & L_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} ; \quad T_{23} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_3 & -s_3 & L_2 \\ 0 & s_3 & c_3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 T_{34} &= \begin{bmatrix} c_4 & -s_4 & 0 & 0 \\ s_4 & c_4 & 0 & L_3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} ; \quad T_{45} = \begin{bmatrix} c_5 & 0 & s_5 & 0 \\ 0 & 1 & 0 & L_4 \\ -s_5 & 0 & c_5 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} ; \quad T_{56} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_6 & -s_6 & L_5 \\ 0 & s_6 & c_6 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 T_{6-ee} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & L_6 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

Now lets multiply these matrices to get the elements of the Jacobian matrix. (I will only show the required elements in the following matrices to build the Jacobian of our 6-DOF manipulator)

$$T_{b1} = \begin{bmatrix} X & X & 0 & X \\ X & X & 0 & X \\ X & X & 1 & X \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{b2} = T_{b1} * T_{12}$$

$$T_{b2} = \begin{bmatrix} c_1 & X & X & X \\ s_1 & X & X & X \\ 0 & X & X & X \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{b3} = T_{b2} * T_{23}$$

$$T_{b3} = \begin{bmatrix} c_1 & X & X & X \\ s_1 & X & X & X \\ 0 & X & X & X \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{b4} = T_{b3} * T_{34}$$

$$T_{b4} = \begin{bmatrix} X & X & s_1 c_2 s_3 + s_1 s_2 c_3 & X \\ X & X & -c_1 c_2 s_3 + c_1 s_2 c_3 & X \\ X & X & -s_2 s_3 + c_2 c_3 & X \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{b5} = T_{b4} * T_{45}$$

$$T_{b5} = \begin{bmatrix} X & -c_1 s_4 - s_1 c_2 c_3 c_4 + s_1 s_2 s_3 c_4 & X & X \\ X & -s_1 s_4 + c_1 c_2 c_3 c_4 - c_1 s_2 s_3 c_4 & X & X \\ X & s_2 c_3 c_4 + c_2 s_3 c_4 & X & X \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{b6} = T_{b5} * T_{56}$$

$$T_{b6} = \begin{bmatrix} C_1 C_4 C_5 - S_1 C_2 C_3 S_4 C_5 + S_1 S_2 S_3 S_4 C_5 - S_1 C_2 S_3 S_5 - S_1 S_2 C_3 S_5 & X & X & X \\ S_1 C_4 C_5 + C_1 C_2 C_3 S_4 C_5 - C_1 S_2 S_3 S_4 C_5 + C_1 C_2 S_3 S_5 + C_1 S_2 C_3 S_5 & X & X & X \\ S_2 C_3 S_4 C_5 + C_2 S_3 S_4 C_5 + S_2 S_3 S_5 - C_2 C_3 C_5 & X & X & X \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{b-ee} = T_{b6} * T_{6-ee}$$

$$T_{b-ee} = \begin{bmatrix} X & X & X & x \\ X & X & X & y \\ X & X & X & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$X = (-C_1 S_4 C_6 - S_1 C_2 C_3 C_4 C_6 + S_1 S_2 S_3 C_4 C_6 + C_1 C_4 S_5 S_6 - S_1 C_2 C_3 S_4 S_5 S_6 + S_1 S_2 S_3 S_4 S_5 S_6 + S_1 C_2 S_3 C_5 S_6 + S_1 S_2 C_3 C_5 S_6) L_6 \\ + (-C_1 S_4 - S_1 C_2 C_3 C_4 + S_1 S_2 S_3 C_4) L_5 + (-C_1 S_4 - S_1 C_2 C_3 C_4 + S_1 S_2 S_3 C_4) L_4 + (-S_1 C_2 C_3 + S_1 S_2 S_3) L_3 - S_1 C_2 L_2$$

$$Y = (-S_1 S_4 C_6 - C_1 C_2 C_3 C_4 C_6 - C_1 S_2 S_3 C_4 C_6 + S_1 C_4 S_5 S_6 + C_1 C_2 C_3 S_4 S_5 S_6 - C_1 S_2 S_3 S_4 S_5 S_6 - C_1 C_2 S_3 C_5 S_6 - C_1 S_2 C_3 C_5 S_6) L_6 \\ + (-S_1 S_4 + C_1 C_2 C_3 C_4 - C_1 S_2 S_3 C_4) L_5 + (-S_1 S_4 + C_1 C_2 C_3 C_4 - C_1 S_2 S_3 C_4) L_4 + (C_1 C_2 C_3 - C_1 S_2 S_3) L_3 + C_1 C_2 L_2$$

$$Z = (S_2 C_3 C_4 C_6 + C_2 S_3 C_4 C_6 + S_2 C_3 S_4 S_5 S_6 + C_2 S_3 S_4 S_5 S_6 - S_2 S_3 C_5 S_6 + C_2 C_3 C_5 S_6) L_6 + (S_2 C_3 C_4 + C_2 S_3 C_4) L_5 \\ (S_2 C_3 C_4 + C_2 S_3 C_4) L_4 + (S_2 C_3 + C_2 S_3) L_3 + S_2 L_2 + L_1 +$$

OMG that was a tedious job. I'm tired now. All the equations required for Jacobian matrix are already up there, kindly find the partial derivatives yourself and arrange all the elements to get the final J matrix. Thank you very much.

$$J = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \frac{\partial x}{\partial q_2} & \frac{\partial x}{\partial q_3} & \frac{\partial x}{\partial q_4} & \frac{\partial x}{\partial q_5} & \frac{\partial x}{\partial q_6} \\ \frac{\partial y}{\partial q_1} & \frac{\partial y}{\partial q_2} & \frac{\partial y}{\partial q_3} & \frac{\partial y}{\partial q_4} & \frac{\partial y}{\partial q_5} & \frac{\partial y}{\partial q_6} \\ \frac{\partial z}{\partial q_1} & \frac{\partial z}{\partial q_2} & \frac{\partial z}{\partial q_3} & \frac{\partial z}{\partial q_4} & \frac{\partial z}{\partial q_5} & \frac{\partial z}{\partial q_6} \\ R_{b1}[:3] & R_{b2}[:1] & R_{b3}[:1] & R_{b4}[:3] & R_{b5}[:2] & R_{b6}[:1] \end{bmatrix}$$

Contact Us

Name

Email

Phone

Type your message here...

Submit

